QLORA: Efficient Finetuning of Quantized LLMs

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, Luke Zettlemoyer

University of Washington

Background: Large Language Models are Transforming the World

- We see LLMs powering search, coding assistants, chatbots, and scientific discovery.
- We finetune base models to adapt to domains, safety, and instruction following.
- We believe access to capable, tailored LLMs impacts education, healthcare, and productivity.

The Problem: Finetuning Very Large LLMs Is Prohibitively Memory-Intensive

- We observe that 16-bit finetuning of 33B–65B models needs hundreds of GBs of GPU RAM.
- We find small labs cannot run high-end finetuning; research and deployment are gated.
- We note that quantization helps for inference but typically breaks training/backprop.

Why This Is Hard: Training Memory Is Dominated by More Than Weights

- We profile memory and find activations, input gradients, and optimizer states dominate peaks.
- We use LoRA to reduce trainable parameters, yet memory spikes can still cause OOM.
- We see long sequences and large batches exacerbate transient peaks.

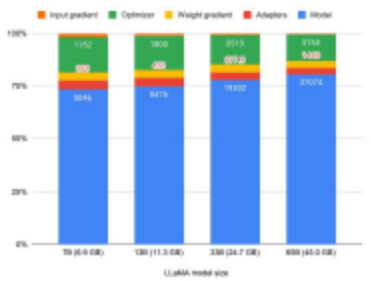

Figure: Takeaway: Activations and optimizer states dominate peak memory; paged optimizers smooth spikes.

We introduce QLORA: Efficient finetuning via 4-bit storage + LoRA

- We backprop through a frozen 4-bit base model into 16-bit LoRA adapters.
- We introduce NF4 and Double Quantization to cut memory without losing accuracy.
- We use paged optimizers to absorb memory spikes, enabling single-GPU 65B finetuning.

We show where QLORA saves memory

- We store base weights at 4-bit and dequantize to BF16 on-the-fly for compute.
- We train only small LoRA adapters while keeping base weights frozen.
- We rely on unified memory paging to handle transient optimizer/activation demands.

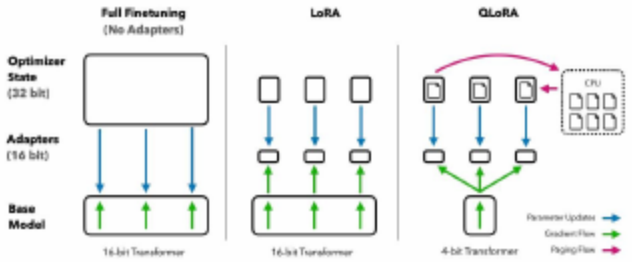

Figure: Takeaway: QLORA cuts memory vs. LoRA by 4-bit storage and paging for spikes.

How we split storage and compute

- We store base model weights in 4-bit NF4 with block-wise scales.
- We dequantize blocks to BF16 on forward/backward for stable compute.
- We backprop to update only LoRA adapter weights (BF16); the base remains frozen.

NF4: A 4-bit Datatype Matched to Weight Distributions

- We observe pretrained LLM weights are near zero-mean normal per hidden unit.
- We use NF4 with quantile-based binning calibrated to N(0,1) for fidelity.
- We find NF4 outperforms FP4/Int4 in perplexity and zero-shot accuracy.
- Takeaway: Aligning the quantizer to per-neuron normal weight statistics improves fidelity.

Figure 5: The crowdsourcing form used by human annotators.

Double Quantization: Shrinking Metadata Overhead

- We first quantize weights in 4-bit blocks with per-block scales.
- We then quantize those scales in 8-bit with larger blocks.
- We reduce overhead from 0.5 to 0.127 bits/param (≈3 GB on 65B).

How we prevent OOM with paged optimizers

- We use NVIDIA Unified Memory for optimizer states via a custom allocator.
- We page states between GPU and host to absorb transient peaks.
- We maintain throughput at tested settings while avoiding OOMs.

Practical LoRA Regimen Matters (backup)

- We apply adapters to all linear layers (attention + FFN).
- We find LoRA rank r less critical once applied broadly; modest dropout helps.
- We use BF16 compute and simple LR schedules across runs.

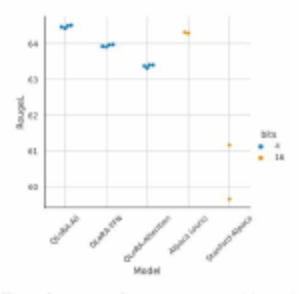

Figure: Takeaway: Applying LoRA to all linear layers matches strong 16-bit baselines on Alpaca.

Our experimental setup and training details (concise)

- We evaluate ¿1,000 finetuned models across LLaMA, T5, and RoBERTa (80M–65B).
- We use instruction datasets including OASST1, Alpaca, FLAN v2, and others.
- We evaluate on GLUE, Super-NI, MMLU, the Vicuna benchmark, and Elo tournaments.
- We use BF16 compute, dataset-length grouping, and constant LR schedules.
- We apply LoRA to all linear layers with moderate ranks and dropout for smaller models.
- We integrate custom CUDA kernels for 4-bit training with Transformers.

We show QLORA matches 16-bit on GLUE and Super-NI

- We match full 16-bit and BF16-LoRA quality across models and scales.
- We confirm quantized finetuning preserves core task performance.

Table: Table 3: Experiments comparing 16-bit BrainFloat (BF16), 8-bit Integer (Int8), 4-bit Float (FP4), and 4-bit NormalFloat (NF4) on GLUE and Super-NaturalInstructions.

Dataset	GLUE (Acc.)	Super-NaturalInstructions (RougeL)						
Model	RoBERTa-large	T5-80M	T5-250M	T5-780M	T5-3B	T5-11B		
BF16	88.6	40.1	42.1	48.0	54.3	62.0		
BF16 replication	88.6	40.0	42.2	47.3	54.9	-		
LoRA BF16	88.8	40.5	42.6	47.1	55.4	60.7		
QLORA Int8	88.8	40.4	42.9	45.4	56.5	60.7		
QLORA FP4	88.6	40.3	42.4	47.5	55.6	60.9		
QLORA NF4 + DQ	-	40.4	42.7	47.7	55.3	60.9		

We find NF4 outperforms FP4/Int4; DQ preserves accuracy

- We observe consistent zero-shot gains for NF4 vs. FP4/Int4.
- We use DQ to trade memory for capacity without accuracy loss.
- We validate that aligning datatype to weight statistics matters.

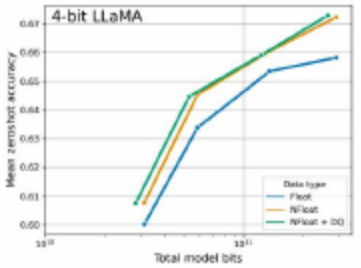

Figure: Takeaway: NF4 yields higher zero-shot accuracy; DQ preserves accuracy while fitting 33B/65B into 24/48 GB.

We match BF16 on MMLU across 7B–65B with NF4+DQ

- FP4 trails by ∼1 point on average.
- We show quantized finetuning scales to large instruction tasks.

Table: Table 4: Mean 5-shot MMLU test accuracy for LLaMA 7-65B models finetuned with adapters on Alpaca and FLAN v2 for different data types.

	Mean 5-shot MMLU Accuracy								
LLaMA Size	7B		13B		33B		65B		Mean
Dataset	Alpaca	FLAN v2	Alpaca	FLAN v2	Alpaca	FLAN v2	Alpaca	FLAN v2	
BFloat16	38.4	45.6	47.2	50.6	57.7	60.5	61.8	62.5	53.0
Float4	37.2	44.0	47.3	50.0	55.9	58.5	61.3	63.3	52.2
NFloat4 + DQ	39.0	44.5	47.5	50.7	57.3	59.2	61.8	63.9	53.1

We achieve near-ChatGPT quality on Vicuna

- We reach ∼99.3% of ChatGPT (GPT-4 judged) with Guanaco-65B.
- We rank just below GPT-4 in Elo tournaments.

Table: Table 1: Elo ratings for a competition between models, averaged for 10,000 random initial orderings.

Model	Size	Elo
GPT-4	-	1348 ± 1
Guanaco 65B	41 GB	1022 ± 1
Guanaco 33B	21 GB	992 ± 1
Vicuna 13B	26 GB	974 ± 1

We reduce memory from ¿780 GB to ¡48 GB for 65B.

- We finetune 65B on a single 48 GB GPU and 33B on 24 GB.
- We avoid OOM at realistic sequence lengths via paged optimizers.
- We attribute the savings to 4-bit storage (NF4), double quantization of scales, and frozen-base LoRA adapters.
- We maintain competitive throughput by paging optimizer states only when transient peaks occur.

Limitations and Failure Cases

- We acknowledge lemon-picked analyses: Guanaco still lags ChatGPT on tricky prompts.
- We find current chatbot benchmarks can mis-rank systems and are not fully trustworthy.
- We note model-based judges are cost-effective but exhibit biases.

Our takeaways

- We democratize finetuning of 33B/65B models on a single GPU.
- We match 16-bit quality using 4-bit storage and LoRA.
- We enable affordable, systematic studies across data, sizes, and tasks.

Baselines and References

- LoRA: Hu et al., 2021. LoRA: Low-Rank Adaptation of Large Language Models (arXiv:2106.09685).
- Vicuna: Chiang et al., 2023. Vicuna: An Open-Source Chatbot.
- ChatGPT (GPT-3.5): OpenAI, 2022. Introducing ChatGPT (blog).
- GPT-4: OpenAI, 2023. GPT-4 Technical Report (arXiv:2303.08774).
- Bard/PaLM 2: Anil et al., 2023. PaLM 2 Technical Report (arXiv:2305.10403); Google Bard announcement (2023).
- RoBERTa: Liu et al., 2019.
- T5: Raffel et al., 2020.
- LLaMA: Touvron et al., 2023.

Future Directions and Broader Impact

- We aim to push precision lower (e.g., 3-bit) while preserving accuracy.
- We will explore other PEFT methods under quantized finetuning.
- We plan privacy-preserving, domain-specialized finetuning, possibly on-device.

Questions & Discussion

- We appreciate your attention!
- We welcome questions and feedback.

Acknowledgments

- We thank collaborators, open-source communities, and compute providers.
- We release code and models, including CUDA kernels for 4-bit training.